КУРСОВОЙ ПРОЕКТ · СИСТЕМНОЕ ПРОГРАММИРОВАНИЕ

ScreenDrop

Локальная передача файлов на Android-экран по QR-коду

Экран — HTTP-сервер. Телефон — браузер.
Без установки приложений и без интернета.

Kotlin · Jetpack Compose · Ktor CIO · TCP/IP · HTTP · MediaStore

---

Выполнил: _____

Группа: _____ · 2026

Сканируйте камерой

откроется веб-отправитель ScreenDrop

ПРОБЛЕМА

Файл на большой экран — это 5 лишних шагов

01

Мессенджеры

Нужен аккаунт, фото и документы сжимаются, зависит от интернета и чужого сервиса.

02

USB и флешки

Физический доступ к экрану, риск заражения, на аудитории это медленно и неудобно.

03

Облачные диски

Внешний сервис, лимиты, приватность документов и лишние авторизации.

Экран сам становится сервером

`QR → браузер → multipart → файл на экране`

Ничего не устанавливаем на телефон и не отправляем данные в интернет.

1 QR-код

0 приложений

0 облаков

АРХИТЕКТУРА

Один APK — и сервер, и приёмник файлов



SessionManager
токен 256 бит · idle 30 мин

NetworkMonitor
ConnectivityManager · IPv4

MainViewModel + UI
state machine · QR

→ Внешнего backend нет: все компоненты работают внутри Android-приложения, данные не покидают локальную сеть.

СЕТЬ

Как телефон находит экран

- **Сценарии.** Два сценария: Personal Hotspot телефона или общая Wi-Fi-сеть.
- **Мониторинг.** ConnectivityManager + NetworkCallback: Wi-Fi/Ethernet, IPv4, VPN исключён.
- **Bind.** Сервер слушает 0.0.0.0:8080 и доступен по локальному IP.
- **Адрес.** В QR — адрес экрана и одноразовый токен сессии.

> ! **Главный риск MVP**
>
> Локальный bind не доказывает доступность с телефона: hotspot и гостевой Wi-Fi могут изолировать клиентов. Проверяем реальным GET/POST.



QR сессии

`http://192.168.43.112:8080/?token=…`

ПРОТОКОЛ

HTTP API и жизненный цикл сессии

GET	/	веб-отправитель (assets)
GET	/api/health	проверка живости
GET	/api/session	статус и токен сессии
POST	/api/upload	multipart-загрузка файла
GET	/api/files	список принятых файлов
DELETE	/api/files/{id}	удаление файла
POST	/api/session/close	закрыть сессию

Upload, files и delete требуют Authorization: Bearer <token>

Сессия

- 256 бит из SecureRandom → base64url
- idle-таймаут 30 минут
- закрытие инвалидирует токен
- новая сессия = новый QR

КОДЫ ОТВЕТА

200 / 201	успех
400 / 401	запрос / токен
413 / 415	размер / формат
429	> 3 загрузок

ЯДРО СИСТЕМЫ

Streaming upload: файл не попадает в RAM



O(1) по памяти

Буфер 64 КиБ: размер файла не влияет на heap

Лимиты и защита

100 МиБ · 3 потока · sanitize · path traversal

Атомарность

Файл виден только после успешной финализации

Обрыв сети → .part удаляется, приёмник возвращается в READY. Неполный файл никогда не считается готовым.

НОВЫЙ РЕЖИМ

Wi-Fi Share: подключить телефон одним сканом

Если гость ещё не в сети — показываем два QR подряд.

1. **Подключение к сети**

   `WIFI:S:<SSID>;T:WPA;P:<пароль>;;`

2. **Передача файла**

   `http://<IP>:8080/?token=<TOKEN>`

3. **Разрешения**

   `CHANGE_WIFI_STATE · NEARBY_WIFI_DEVICES (13+)`

◐ **Статус: прототип**

Генерация QR и чтение SSID реализованы. Активация hotspot и выдача пароля — следующий шаг.



Подключение к Wi-Fi

QR №1



Передача файла

QR №2

Формат WIFI: понимают стандартные камеры Android и iOS — отдельное приложение не нужно.

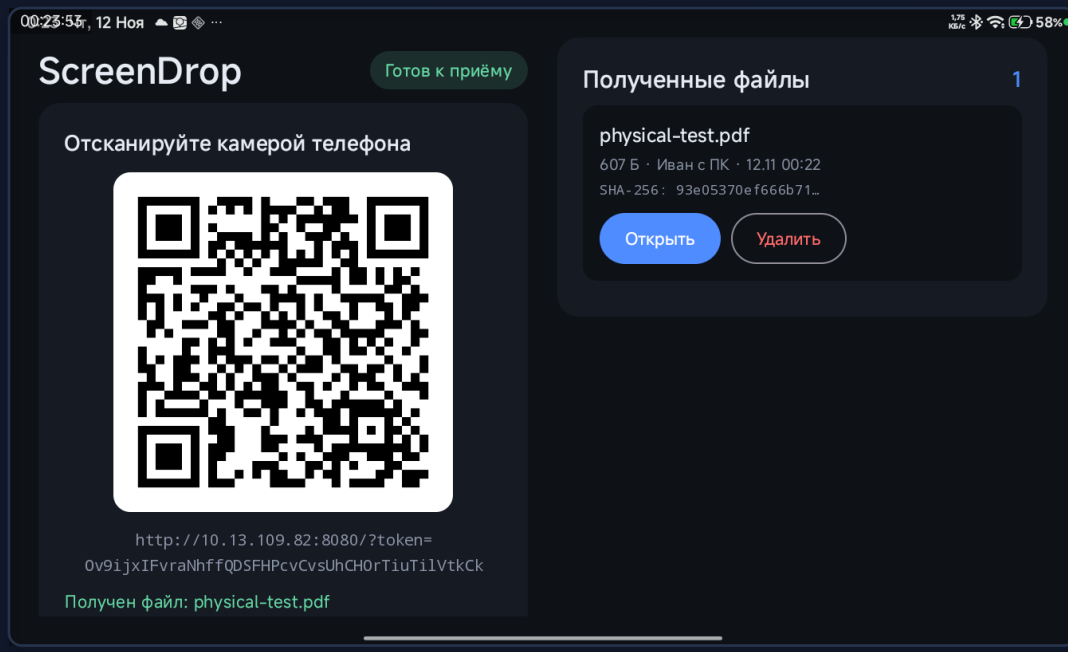
ПРОВЕРКА

37 тестов и физический прогон

37 / 37

JVM-тестов зелёные

- **Session:** токен, истечение, закрытие
- **Upload:** лимиты, имена, path traversal, SHA-256
- **Storage:** финализация, metadata, удаление
- **Server:** роуты, авторизация, 401/413/415/429



Планшет: QR сессии и принятый файл в списке

Физический тест: PC → планшет

- PDF загружен: 201, SHA-256 совпал, файл в списке
- hotspot и общая Wi-Fi: QR перегенерирован, сессия жива
- 401 / 415 / 429 / 404 — ожидаемые отказы

ОГРАНИЧЕНИЯ

Что MVP сознательно не решает

HTTP без TLS

Токен и файл видны в локальном трафике → только доверенная сеть.

Client isolation

Гостевой Wi-Fi может блокировать связь → проверка доступности.

Фоновая работа

Сервер живёт при открытом UI → Foreground Service.

Нет resumable

Обрыв = повторная загрузка → Range-запросы, P2.

Зависимость от viewer

Открытие файла — через ACTION_VIEW и установленное приложение.

Один приёмник

Нет multi-room и автообнаружения → desktop, mDNS, несколько экранов.

Честные границы проекта — часть инженерного результата: мы знаем, где решение работает, а где нет.

ИТОГИ

Что получилось

1. **Работающий MVP**

   APK, 37 тестов, реальная передача PC → планшет, файлы в Documents/ScreenDrop.

2. **Техническая база**

   Встроенный Ktor, streaming I/O, атомарная запись, QR-сессии и лимиты.

3. **Понятный UX**

   Скан → браузер → файл на экране. Без установки приложения на телефон.

Дальше:

Foreground Service · TLS · resumable uploads · hotspot-режим



github.com/zt-a/screendrop

исходники и APK в репозитории

Спасибо! Вопросы?